Design Patterns în Java — Teorie

Modul construit lecție cu lecție · corespondent exercițiilor din repo

Lecția 0 — Ce este un design pattern (puntea de la interfețe)

De unde venim

În `java-abstractizare` ai scris interfața `Duplicabil`, cu o singură metodă: `Duplicabil interfaceDuplicate();`. Au semnat-o trei ierarhii care n-au nicio rudă comună: figurile din `formeGeometrice`, intervalele din `OrarScheduler` (`Pauza`, `Examen`) și `Orar` însuși. Codul care copiază nu întreabă CE e obiectul, ci doar dacă ȘTIE să se copieze.

Tot acolo ai pus teza: **`abstract` răspunde la „ce ești”, `interface` la „ce poți face”**. Și ai folosit un contract scris de altcineva: `Comparable`. Clasa ta implementează `compareTo`, iar `Collections.sort` o sortează fără să știe nimic despre ea, doar că respectă contractul.

Ține minte propoziția asta, pentru că e singura idee din tot modulul:

> **Depinzi de contract, nu de clasă.**

Toate pattern-urile pe care le facem sunt aceeași idee, aplicată la trei întrebări diferite.

Ce e, de fapt, un pattern

Un **design pattern** nu e o bibliotecă, o clasă gata făcută sau o sintaxă nouă. E un **nume pentru o soluție care se repetă** la o problemă care se repetă. Cineva a lovit aceeași problemă de o mie de ori, a găsit forma care merge și i-a dat un nume, ca să putem vorbi despre ea în două cuvinte în loc de zece minute.

Beneficiul e dublu:

- **Nu reinventezi** — când recunoști problema, știi deja forma soluției.
- **Comunici scurt** — spui „aici e un Strategy” și toată echipa înțelege structura, fără să citească fiecare linie.

Riscul, pe care îl vom evita: pattern-ul aplicat unde nu trebuie. Un pattern rezolvă o durere concretă. **Fără durere, fără pattern** — altfel adaugi complexitate degeaba.

Firul logic al modulului — trei întrebări

Grup	Întrebarea la care răspunde	Pattern-uri
Comportament	*cine se comportă și cum?*	Strategy, Observer, State
Structură	*cum compun obiectele?*	Decorator, Adapter
Creare	*cine și cum creează obiectele?*	Factory Method, Builder, Singleton

Modulul de acum ia patru dintre ele. Începem cu **comportamentul**, pentru că e cel mai aproape de interfețe. Primul pattern — Strategy — e practic un pas mic peste ce știi deja.

Harta modulului

Lecția	Pattern	Ghidat	Pe cont propriu
1	Strategy	`ex1` — cost de livrare	`ex3` — export de raport
2	Observer	`ex2` — starea comenzii	`ex4` — stație meteo
3	Factory Method	`ex5` — încărcarea utilizatorilor	—
4	Decorator	`ex6` — notificări	—

Ordinea de lucru: `ex1` → `ex2` → `ex3` → `ex4` → `ex5` → `ex6`. Caietul `EXERCITII.pdf` are exerciții în plus pe fiecare pattern. La fiecare pattern găsești și secțiunea **„Unde îl aplici în aplicația ta”**: locul concret din online-school unde se potrivește — sau, cinstit, de ce nu se potrivește încă.

Lecția 1 — Strategy

> **Un cuvânt:** comportament interschimbabil. Închizi fiecare variantă de „cum se face ceva” într-o clasă separată care semnează același contract și o schimbi când vrei — chiar la runtime.

Problema (exercițiul `ex1`: costul de livrare)

Un magazin calculează costul de livrare al unei comenzi. Costul depinde de metodă: standard, express, promoție gratuită. Prima variantă la care se gândește oricine:

```
public enum TipLivrare { STANDARD, EXPRESS, GRATUITA }

public double calculeazaCost(TipLivrare tip, double greutate,
                             double distanta) {
    switch (tip) {
        case STANDARD:
            return 5 + 0.5 * greutate + 0.1 * distanta;
        case EXPRESS:
            return 12 + 1.0 * greutate + 0.25 * distanta;
        case GRATUITA:
            return 0;
        default:
            throw new IllegalArgumentException("Unknown delivery type");
    }
}
```

Merge. Și e o capcană. Întreabă-te ce faci când firma adaugă `LivrareInternationala`:

- deschizi metoda asta și adaugi un `case`;
- probabil mai există un `switch` la fel în alt loc (afișare, facturare) — îl deschizi și pe ăla;
- fiecare `switch` uitat = un bug tăcut.

Aici intră o regulă nouă, **open/closed**: *o clasă ar trebui să fie deschisă la extindere, dar închisă la modificare*. Adică adaugi comportament nou scriind cod nou, nu editând cod care deja merge. Ai simțit-o deja în fișa de interfețe: un `Romb` cu arie, adăugat fără să atingi codul care adună ariile. Cu `switch`-ul de mai sus, ca să extinzi (metodă nouă de livrare) ești **obligat să modifici**. `switch`-ul pe tip e mirosul clasic care strigă „aici lipsește un pattern”.

Ideea

Întoarce lucrurile pe dos. În loc ca o metodă să știe TOATE variantele și să aleagă cu `switch`, **fiecare variantă devine o clasă** care știe DOAR de ea și semnează un contract comun:

```
public interface LivrareStrategie {
    String getNume();
    double calculeazaCost(double greutateKg, double distantaKm);
}
```

`Comanda` nu mai conține nicio formulă și niciun `switch`. Ține o `LivrareStrategie` și o **întreabă**:

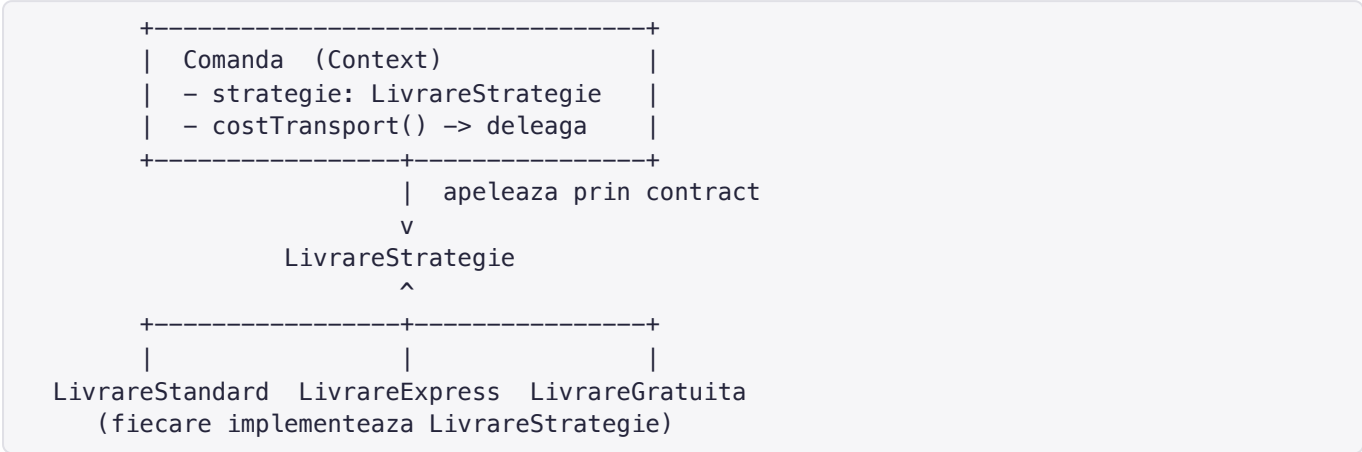
```
public double costTransport() {
    return strategie.calculeazaCost(greutateKg, distantaKm);
}
```

O metodă nouă de livrare = **o clasă nouă**, atât. Nu atingi `Comanda`, nu atingi celelalte strategii. Închis la modificare, deschis la extindere — de data asta chiar.

Cele trei roluri

Rol	În exercițiu	Ce face
Strategy (contractul)	`LivrareStrategie`	declară operația interschimbabilă

ConcreteStrategy	`LivrareStandard`, `LivrareExpress`, `LivrareGratuita`	fiecare, o singură variantă a algoritmului
Context	`Comanda`	ține o strategie, deleagă spre ea, o poate schimba



Mecanismul de dedesubt

Nu e magie — sunt trei lucruri pe care le știi deja, puse împreună:

1. **Polimorfism prin contract.** `strategie.calculeazaCost(...)` cheamă implementarea clasei reale din spatele interfeței. Exact ce făcea `interfaceDuplicate()` chemat pe un `Duplicabil`: se copia o figură sau un `Examen`, după obiectul real.
2. **Delegare.** `Comanda` nu *face* calculul, îl *pasează* mai departe. Contextul e un dispecer, nu un executant.
3. **Injectare.** Strategia intră din afară (prin constructor sau `schimbaStrategie`), nu e creată înăuntru. De-aia o poți schimba la runtime — și de-aia contextul nu depinde de nicio clasă concretă.

Detaliul care le leagă: `calculeazaCost` primește greutatea și distanța ca **parametri**, nu le ține strategia în câmpuri. Așa aceeași instanță `LivrareStandard` servește orice comandă — strategia e *fără stare*, deci refolosibilă și fără surprize. (Vezi întrebarea din bonusul exercițiului.)

Când îl folosești

- Ai **mai multe variante** ale aceluiași lucru (calcul, sortare, validare, format de export) și vrei să comuți între ele.
- Vezi un **`switch`/lanț de `if`** pe un „tip” care se tot repetă prin cod.
- Vrei să **adaugi variante** fără să atingi codul existent.
- Vrei să poți schimba comportamentul **la runtime**, nu doar la compilare.

Când NU îl folosești

- Ai **o singură variantă** și nu se întrevăd altele. Un `if` simplu e mai onest decât trei clase și o interfață.
- Variantele diferă printr-o **singură valoare**, nu printr-un algoritm (atunci e un parametru, nu o strategie).
- Pattern-ul aplicat de dragul pattern-ului adaugă doar ceremonie. Amintește-ți: fără durere, fără pattern.

Capcane frecvente

- **Context cu `switch` rămas.** Dacă `Comanda` tot decide cu `if` care strategie să folosească, n-ai mutat problema — ai dublat-o. De-aia constrângerea din exercițiu interzice `if` / `switch` pe tip în `Comanda`.
- **Strategie cu stare de comandă.** Dacă `LivrareStandard` ține greutatea în câmp, n-o mai poți refolosi între comenzi. Ține strategia fără stare; datele vin ca parametri.
- **Prea multe strategii minuscule.** Dacă variantele diferă cu o linie, poate era destul o lambda sau un parametru.

Unde îl aplici în aplicația ta (online-school)

Locul: `View.java`, metodele `play()` (linia 38) și `playTeacher()` (linia 232). Sunt două meniuri, fiecare cu `switch`-ul lui pe opțiunea aleasă.

Mirosul. `switch`-ul pe *opțiune* nu e problema: utilizatorul tastează un număr, iar un `switch` pe ce a tastat e cinstit. Problema e alegerea **meniului**. Ce meniu primești depinde de rol (student sau profesor), dar azi meniul de

profesor e opțiunea 8 din meniul de student, deci îl vede oricine. Metoda care ar trebui să decidă, `checkUserType()` (linia 349), nu e chemată de nicăieri, iar utilizatorul e un `Teacher` scris de mână în constructor (linia 26). Ai deja două variante ale aceluiași lucru („meniul utilizatorului") și o decizie pe tip care nu are unde să stea. Aici se potrivește Strategy.

Forma, pe clasele tale:

```
public interface MeniuRol {
    void afiseazaOptiuni();
    void executa(int optiune);
}

public class MeniuStudent implements MeniuRol { ... }
public class MeniuTeacher implements MeniuRol { ... }
```

```
public class View {
    private MeniuRol meniu;

    public void play() {
        int alegere = -1;
        while (alegere != 0) {
            meniu.afiseazaOptiuni();
            alegere = Integer.parseInt(scanner.nextLine());
            meniu.executa(alegere);
        }
    }
}
```

Rol	În online-school
Strategy	`MeniuRol`
ConcreteStrategy	`MeniuStudent` (cărți, înscrieri), `MeniuTeacher` (cursurile mele, studenții mei)
Context	`View`: ține meniul și îl întreabă; nu mai știe ce opțiuni există

Cine alege strategia? O singură decizie, într-un singur loc: acolo unde afli cine e utilizatorul (la login). Nu în `play()`. Dacă `View` ajunge să întrebe „ești profesor?" la fiecare pas, ai capcana „context cu `switch` rămas".

Precondiția, cinstit: ca să alegi meniul după rol, utilizatorul trebuie să aibă rolul corect. Azi nu-l are. Asta e lecția 3 (Factory Method), iar înainte de ea vin bug-urile din review. Deci Strategy pe meniu e al doilea pas în aplicația ta, nu primul.

Practică

`ex3` (export de raport) e același pattern pe un domeniu nou, fără pași. Îl faci după `ex2`.

Legătura cu ce urmează

În bonusul exercițiului scrii `LivrareCuReducere`: o strategie care primește **altă strategie** în constructor și îi ajustează rezultatul. O strategie care împachetează o strategie — asta e deja un **Decorator** (lecția 4). Strategy ți l-a arătat gratis; acolo doar îi punem numele.

Înainte de el, lecția 2 rămâne pe aceeași `Comanda` și schimbă întrebarea: nu *cum se calculează*, ci *cine reacționează*.

Lecția 2 — Observer

> **Un cuvânt:** un eveniment, N reacții. O sursă își schimbă starea și îi anunță pe toți cei abonați — fără să știe cine sunt sau ce fac.

Problema (exercițiul `ex2`: starea comenzii)

Aceeași comandă, dar acum urmărim starea ei: `Plasata → Expediata → Livrata`. La fiecare schimbare reacționează mai multe părți: clientul primește email, se scrie în jurnal, depozitul se pregătește. Prima variantă:

```
public void schimbaStare(String stareNoua) {
    stare = stareNoua;
    email.trimite(stareNoua);
    jurnal.scrie(stareNoua);
    depozit.pregateste(stareNoua);
}
```

`Comanda` ajunge să cunoască fiecare parte pe nume. Marketingul cere o notificare push → redeschizi `schimbaStare` și mai adaugi o linie. O clasă care ar trebui să se ocupe de *comandă* știe acum de email, jurnal, depozit, push. E aceeași durere de open/closed ca la Strategy, pe altă axă: acolo se schimba **cum se calculează**, aici se schimbă **cine reacționează**.

Ideea

Inversezi dependența. `Comanda` nu mai cunoaște nicio parte concretă. Ține o listă de observatori care au semnat un contract, iar când se schimbă starea îi anunță pe toți, la fel:

```
public interface Observator {
    void actualizeaza(String stareNoua);
}
```

```
private final List<Observator> observatori = new ArrayList<>();

public void aboneaza(Observator observator) {
    observatori.add(observator);
}

public void schimbaStare(String stareNoua) {
    stare = stareNoua;
    for (Observator observator : observatori) {
        observator.actualizeaza(stareNoua);
    }
}
```

O parte nouă care vrea să reacționeze = o clasă nouă care implementează `Observator` și se abonează. `Comanda` rămâne neatinsă.

Cele trei roluri

Rol	În exercițiu	Ce face
Subject (sursa)	`Comanda`	ține observatorii, își schimbă starea, îi anunță pe toți
Observer (contractul)	`Observator`	declară `actualizeaza(...)`
ConcreteObserver	`NotificatorEmail`, `JurnalLivrare`, `PanouDepozit`	fiecare reacționează în felul lui

```
                Comanda  (Subject)
                - observatori: List<Observator>
                - schimbaStare() -> anunta pe toti
                        |
          +-------------+-------------+
          v             v             v
  NotificatorEmail  JurnalLivrare  PanouDepozit
        (fiecare implementeaza Observator)
```

Mecanismul de dedesubt

1. **O listă de contracte.** `List<Observator>` e același lucru ca lista ta mixtă de `Duplicabil`: obiecte de clase diferite, ținute împreună pentru că știu toate să facă un lucru.

2. **Sursa împinge.** Nu observatorii întreabă „s-a schimbat ceva?”, ci sursa îi cheamă pe ei, în momentul schimbării. De-aia `actualizeaza` primește noua stare ca parametru.
3. **Abonare la runtime.** Lista se umple și se golește cât rulează programul (`aboneaza` / `dezaboneaza`). Un observator abonat mai târziu vede doar schimbările de după.

Strategy vs Observer — aceeași unealtă, altă direcție

	Strategy (ex1)	Observer (ex2)
Câți colaboratori	UNUL (strategia curentă)	MULȚI (toți observatorii)
Ce face contextul	îl întreabă: „cât costă?”	îi anunță: „s-a schimbat starea”
Așteaptă răspuns?	DA — primește un cost înapoi	NU — anunță și merge mai departe

„A întreba” vs „a anunța” — asta e distincția de fixat. Observer nu așteaptă nimic înapoi; de-aia `actualizeaza` întoarce `void`.

Când îl folosești

- O schimbare într-un loc trebuie să declanșeze reacții în **mai multe** locuri independente.
- Vrei să adaugi sau să scoți reacții fără să atingi sursa.
- Reacțiile nu depind una de alta și nu întorc nimic sursei.

Când NU îl folosești

- Ai **un singur** ascultător și va rămâne unul — un apel direct e mai clar.
- Ai nevoie de un **răspuns** de la fiecare parte (atunci nu e Observer).
- Ordinea reacțiilor contează strict sau una depinde de alta.

Capcane frecvente

- **Subject care cunoaște observatorii concreți.** Dacă în `schimbaStare` apare `instanceof NotificatorEmail`, ai ratat pattern-ul: sursa trebuie să fie oarbă la tipuri.
- **Lista modificată în timpul anunțului.** Dacă un observator se dezabonează din interiorul lui `actualizeaza`, bucla `for` de pe listă aruncă `ConcurrentModificationException`. Abonarea și dezabonarea se fac din afara anunțului.
- **Notificare fără schimbare reală.** Anunță doar când starea chiar s-a schimbat, altfel observatorii reacționează degeaba.

Unde îl aplici în aplicația ta (online-school)

Răspunsul cinstit: azi, nicăieri. Candidatul firesc e înscrierea la curs: `View.registerForCourse()` (linia 121), care cheamă `EnrollmentRepository.addEnrollment()` (linia 30). Dar la o înscriere se întâmplă azi **un singur lucru**: se salvează și se afișează „enrolled”. Un singur ascultător înseamnă apel direct, nu Observer. În plus, azi nicio opțiune din meniu nu cheamă `registerForCourse()`.

Când devine natural: în ziua în care o înscriere trebuie să declanșeze **a doua** reacție, independentă de prima. Exemple din aplicația ta: la prima înscriere i se emite studentului cardul (`CardStudent` există, dar `cardRepository` nu e folosit nicăieri), profesorul cursului e anunțat, se scade un loc disponibil. Atunci forma ar fi:

```
public interface ObservatorInscriere {
    void laInscriere(Enrollment inscriere);
}

public class EmitereCard implements ObservatorInscriere { ... }
public class AnuntProfesor implements ObservatorInscriere { ... }
```

Sursa evenimentului e locul care **face** înscrierea (azi `View`), nu repository-ul: `EnrollmentRepository` doar salvează. Regula de recunoaștere: **două reacții independente la același eveniment = semnalul. Una = apel direct.**

Practică

`ex4` (stație meteo) e Observer pe un domeniu nou, fără pași. Aici observatorii au și **stare proprie** (statistica se acumulează).

Legătura cu ce urmează

Observer pune reacțiile **alături** de sursă. Mai târziu, Decorator le va pune **în jurul** unui obiect. Dar înainte de el vine o întrebare de alt fel: nu cine se comportă, ci **cine naște obiectele**. Și aici ai deja durerea, în aplicația ta.

Lecția 3 — Factory Method

> **Un cuvânt:** creare delegată. Muți `new` din codul care decide în clase care știu fiecare să nască un singur lucru.

Notă de parcurs. La lecția 0, Factory Method stătea în grupul *Creare*, după State. Îl luăm acum pentru că l-ai lovit deja, singur, în `UserRepository` din online-school. Un pattern se învață cel mai bine când doare.

Problema (exercițiul `ex5`: încărcarea utilizatorilor)

Ai un fișier în care fiecare linie începe cu un tip:

```
STUDENT,Ana,Popescu,anul 2
PROFESOR,Ion,Ionescu,Matematica
STUDENT,Vlad,Marin,anul 1
```

Pentru fiecare linie trebuie construit obiectul potrivit: `Student` sau `Profesor`. Prima variantă:

```
for (String linie : linii) {
    String[] campuri = linie.split(",");
    switch (campuri[0]) {
        case "STUDENT":
            utilizatori.add(new Student(campuri[1], campuri[2],
                                        campuri[3]));
            break;
        case "PROFESOR":
            utilizatori.add(new Profesor(campuri[1], campuri[2],
                                         campuri[3]));
            break;
        default:
            throw new IllegalArgumentException("Unknown user type");
    }
}
```

Merge. Și e exact `switch`-ul de la lecția 1 — aceeași durere, alt loc: ca să adaugi `ADMIN`, redeschizi o metodă care mergea. Dar de data asta Strategy **nu** te salvează, și merită înțeles de ce.

De ce polimorfismul NU rezolvă asta

Uită-te la `UserRepository.loadData()`, linia 126:

```
users.add(new User(line));
```

Ai `Teacher` și `Student`, fiecare cu `getRol()` suprascris. Deci polimorfismul ar trebui să aleagă rolul corect, nu? Nu. Întreabă-te **ce tip are obiectul**: e un `User`, l-ai scris tu acolo. Apelul virtual se duce la tipul **real** al obiectului, deci `getRol()` întoarce `"USER"` pentru toată lumea, iar `findUserTypeById()` (linia 41), cu `instanceof Teacher`, nu găsește niciodată un profesor încărcat din fișier. Override-urile din `Teacher` nu rulează nici măcar o dată.

> **Polimorfismul alege ce METODĂ rulează pe un obiect care există deja. Nu poate alege ce CLASĂ se construiește.**

Când ajungi la `new`, decizia e deja luată: ai scris numele clasei în cod. Polimorfismul intră în scenă *după*. „Cine decide ce clasă se naște” e altă întrebare decât „cine decide cum se comportă”, și are nevoie de alt pattern.

Ideea

Dacă `new Student(...)` nu poate fi ales polimorfic, ascunde-l în spatele a ceva care **poate**: un obiect a cărui singură treabă e să construiască.

```
public interface FabricaUtilizator {
    String getTip();
    Utilizator creeaza(String[] campuri);
}
```

`FabricaStudent` știe să nască doar `Student`; `FabricaProfesor`, doar `Profesor`. Fiecare are `new`-ul ei, în clasa ei. Cel care încarcă fișierul nu mai are niciun `switch` și niciun `new` de model:

```
for (FabricaUtilizator fabrica : fabrici) {
    if (fabrica.getTip().equals(campuri[0])) {
        return fabrica.creeaza(campuri);
    }
}
```

Observă ce e `if`-ul ăsta: o comparație între **două șiruri de date**, nu o verificare de tip. Un tip nou de utilizator = **o clasă-model nouă + o fabrică nouă**, și zero linii modificate în încărcător.

Rolurile

Rol	În exercițiu	Ce face
Product	`Utilizator` (abstract)	ce se construiește, văzut prin contract
ConcreteProduct	`Student`, `Profesor`	produsele reale
Creator	`FabricaUtilizator`	declară metoda care naște un produs
ConcreteCreator	`FabricaStudent`, `FabricaProfesor`	fiecare, un singur `new`
Client	`IncarcatorUtilizatori`	cere un produs, fără să știe ce clasă primește

```
  IncarcatorUtilizatori  (Client)
      - fabrici: List<FabricaUtilizator>
      - niciun "new Student" inauntru
                |
                v  cere prin contract
         FabricaUtilizator              Utilizator
                ^                           ^
         +------+--------+          +--------+-------+
         |               |          |                |
  FabricaStudent   FabricaProfesor  Student        Profesor

  FabricaStudent.creeaza()   -->  new Student(...)
  FabricaProfesor.creeaza()  -->  new Profesor(...)
```

Fiecare fabrică e legată de exact un produs. Asta e toată ideea: perechea (cine creează, ce creează) e închisă într-o clasă.

Mecanismul de dedesubt

1. **`new` e o decizie luată la scriere, nu la rulare.** De-aia nu poate fi făcută polimorfic direct. Singura soluție e s-o *muți* într-un obiect care poate fi ales la rulare.
2. **Indirecție.** Nu chemi constructorul, chemi pe cineva care îl cheamă pentru tine. Exact ce făcea `Comanda` cu strategia, doar că acolo delegai un *calcul*, aici delegi o *naștere*.
3. **Contractul ca tip de retur.** `creeaza` întoarce `Utilizator`, nu `Student`. Clientul folosește produsul doar prin `descriere()`. Dacă după fabrică urmează un cast sau un `instanceof`, ai mutat `switch`-ul, nu l-ai eliminat.

Factory Method vs Simple Factory

Numele se folosesc amestecat, ca să nu te încurci când citești pe net:

- Ce faci în `ex5` — un contract de fabrică, mai multe implementări, alese la rulare — e forma pe care o vei folosi cel mai des. Unii o numesc **Simple Factory**.
- **Factory Method** în forma din carte (GoF) e mai strâmtă: o clasă de bază face o treabă mai mare și lasă o metodă abstractă prin care **subclasele decid produsul**. E exact tiparul pe care l-ai scris în abstractizare: `afisare()` scrisă o dată, care cheamă o metodă abstractă.

Ideea e aceeași: **muți `new`-ul în spatele unui contract**. Diferă doar unde stă metoda.

Când îl folosești

- Trebuie să **alegi clasa la rulare**, după o valoare (un tip din fișier, o setare, o comandă a utilizatorului).
- Ai un `switch`/lanț de `if` care se termină în `new` — mirosul clasic.
- Vrei să adaugi un tip nou fără să atingi codul care le folosește.

Când NU îl folosești

- Ai **o singură clasă** de construit: `new Student(...)` direct e mai onest decât o fabrică cu o implementare.
- Tipul e cunoscut la compilare și construcția e banală.
- Problema nu e „ce clasă”, ci „cum umplu un obiect cu multe câmpuri opționale” — pentru asta există **Builder**.

Capcane frecvente

- **Cast după fabrică.** `(Student) fabrica.creeaza(...)` în client = problema mutată, nu rezolvată.
- **`switch` mutat în fabrică.** O singură clasă `FabricaUtilizatori` cu `switch` înăuntru e mai ordonată, dar tot se redeschide la fiecare tip nou. Câte tipuri, atâtea fabrici.
- **`new` uitat în client.** Caută cuvântul `new` în încărcător. Dacă apare un model concret, pattern-ul nu e complet.

Unde îl aplici în aplicația ta (online-school)

Locul: `UserRepository.loadData()`, linia 126, cu `new User(line)` pentru orice linie. **Mirosul:** `new` hardcodat acolo unde clasa ar trebui aleasă după date; la fel în `View`, linia 26, unde un `Teacher` e scris de mână. Aici Factory Method e cel mai natural pattern din toată aplicația ta.

Forma, pe clasele tale:

```
public interface FabricaUser {
    String getRol();
    User creeaza(String linie);
}

public class FabricaStudent implements FabricaUser { ... }
public class FabricaTeacher implements FabricaUser { ... }
```

```
public class UserRepository {
    private final List<FabricaUser> fabrici;

    public UserRepository(List<FabricaUser> fabrici) {
        this.fabrici = fabrici;
        loadData();
    }
}
```

În `loadData()` cauți fabrica după rol, exact ca în `ex5`. Ce rămâne de decis de tine, în runda de review, sunt două reguli fără de care fabrica n-are ce citi:

- **Linia trebuie să spună singură ce tip e.** Uită-te la `users.txt`: de unde ar afla `loadData()` că o linie e de profesor?
- **Ce scrie `save()` trebuie să poată citi `loadData()`.** Salvează un profesor, repornește aplicația și verifică dacă se încarcă înapoi la fel.

Al doilea loc, forma din carte. Cele 5 repository-uri (Book, Course, Enrollment, card, User) au `loadData()` și `save()` copiate aproape identic; diferă doar `new X(line)` și mesajul de eroare. Asta e forma GoF: o clasă de bază care scrie citirea o singură dată și lasă subclaselor doar metoda care naște obiectul:

```
public abstract class RepositoryFisier<T> {
    protected abstract T dinLinie(String linie);
}
```

E pasul următor, nu primul: cere o clasă generică (`<T>`), pe care n-ai scris-o încă. Întâi fabricile pentru `User`.

Legătura cu ce urmează

Factory Method răspunde la „**ce** clasă construiesc". Lecția 4 răspunde la altceva: cum **adaugi** comportament peste un obiect care există deja, fără să-i atingi clasa. Ai văzut-o deja în bonusul de la `ex1`.

Lecția 4 — Decorator

> **Un cuvânt:** comportament adăugat prin împachetare. O clasă care semnează contractul **și ține înăuntru un alt obiect cu același contract**, îl cheamă și îi ajustează intrarea sau rezultatul. Se pot stivui.

Problema (exercițiul `ex6`: notificări)

O aplicație trimite notificări pe email sau SMS. Peste canal vrei să poți adăuga, în orice combinație: un **prefix** (`[URGENT] `), o **trunchiere** la o lungime maximă, o **jurnalizare** a ce se trimite. Prima variantă la care se gândește oricine e o clasă pentru fiecare combinație:

```
SmsCuPrefix, SmsCuTrunchiere, SmsCuPrefixSiTrunchiere,
SmsCuJurnal, SmsCuPrefixSiJurnal, ... EmailCuPrefix, ...
```

Trei adăugiri pe un canal = opt combinații, deci opt clase. Patru adăugiri = șaisprezece. Pe două canale, dublu. Iar `SmsCuPrefixSiTrunchiere` copiază codul din `SmsCuPrefix` și din `SmsCuTrunchiere`.

Ideea

Fiecare adăugire devine **o clasă care semnează același contract și ține înăuntru un obiect cu același contract**. Face partea ei, apoi îi dă mai departe celui dinăuntru:

```
public interface Notificator {
    String getNume();
    void trimite(String mesaj);
}
```

```
public class CuPrefix implements Notificator {
    private final String prefix;
    private final Notificator interior;

    @Override
    public void trimite(String mesaj) {
        interior.trimite(prefix + mesaj);
    }
}
```

Combinațiile nu mai sunt clase, sunt **obiecte puse unul în altul**, la rulare:

```
Notificator n = new CuPrefix("[URGENT] ",
        new CuTrunchiere(30, new NotificatorSms()));
```

Cele trei roluri

Rol	În exercițiu	Ce face
Component (contractul)	`Notificator`	operația pe care o semnează și cel decorat, și decoratorul
ConcreteComponent	`NotificatorEmail`, `NotificatorSms`	fac treaba de bază, nu împachetează pe nimeni
Decorator	`CuPrefix`, `CuTrunchiere`, `CuJurnal`	semnează contractul, ține un `Notificator`, adaugă ceva și deleagă

```
apelantul  -- vede doar Notificator
     |
     v
CuPrefix         (decorator)  pune "[URGENT] " in fata
     |
     v
CuTrunchiere     (decorator)  taie la 30 de caractere
     |
     v
NotificatorSms   (component)  trimite efectiv
```

De ce merită un nume separat de Strategy

	Strategy	Decorator
Ce ține clasa	contextul ține **strategia**	decoratorul ține **un obiect de același tip cu el**
Câte niveluri	unul	oricâte, stivuite
Cine îl vede	contextul știe că are o strategie	apelantul **nu știe** că e împachetat

Mecanismul de dedesubt

1. **Aceeași interfață în două roluri.** Decoratorul e în același timp *implementare* a contractului și *client* al lui. De-aia poate sta oriunde stă un obiect obișnuit — inclusiv în alt decorator.
2. **Compunere la runtime, nu moștenire.** Trei decoratori dau opt combinații fără nicio clasă în plus.
3. **Ordinea contează.** Fiecare decorator vede doar ce i-a dat cel din afara lui. Cum citești lanțul depinde de *când* lucrează decoratorul:
   - **înainte** să delege (`trimite`: prefix, tăiere, jurnal) — citești **din afară spre înăuntru**. În `CuJurnal(CuTrunchiere(30, sms))`, jurnalul e primul, deci vede mesajul **întreg**; în `CuTrunchiere(30, CuJurnal(sms))` vede mesajul deja **tăiat**;
   - **după** ce deleagă (un cost întors, ca la `LivrareCuReducere`) — citești **dinăuntru spre afară**: întâi se calculează costul de bază, apoi reducerea.
4. **Numele se compune.** Un decorator nu-și întoarce un nume fix, ci îl construiește din cel dinăuntru: `interior.getNume() + " + jurnal"`. Așa `getNume()` îți arată tot lanțul: `SMS + trunchiere + jurnal`.

Când îl folosești

- Vrei să **adaugi** ceva peste un comportament existent, fără să atingi clasa care îl face.
- Ai nevoie de **combinații** de adăugiri, nu de o listă fixă.
- Adăugirea e transversală: jurnalizare, validare, limitare, cache.

Când NU îl folosești

- Ai **o singură** adăugire și nu se întrevăd altele: pune-o în clasa de bază.
- Adăugirea trebuie să știe **ce anume** decorează (`instanceof NotificatorSms`) — atunci nu e decorator.
- Nu există un contract comun. Să inventezi o interfață doar ca să ai ce decora e pattern forțat.
- Vrei ca mai mulți să reacționeze la un eveniment — aia e Observer. Decorator schimbă *ce face* operația; Observer *anunță* că s-a întâmplat ceva.

Capcane frecvente

- **Decorator care nu deleagă.** Dacă în `trimite` nu apare `interior.trimite(...)`, ai scris o clasă obișnuită cu un câmp nefolosit.
- **Nume fix.** `return "Cu jurnal";` ascunde tot ce e dedesubt. Compune-l.
- **Ordine presupusă.** Dacă rezultatul depinde de ordine (și depinde), rulează ambele ordini, nu doar pe cea care-ți iese bine.

Unde îl aplici în aplicația ta (online-school)

Răspunsul cinstit: azi, nicăieri. Decorator are nevoie de un contract pe care îl semnează și obiectul de bază, și cel care îl împachetează. În online-school nu există niciunul: repository-urile sunt clase concrete, chemate direct

din `View`, și nu există notificări. Să scrii o interfață doar ca s-o poți decora ar fi exact „pattern de dragul pattern-ului”.

Când devine natural:

- după ce repository-urile au o bază comună (forma GoF din lecția 3) și vrei ceva **în plus** la fiecare `save()` — un jurnal, o copie de siguranță — fără să atingi cele 5 clase;
- dacă apare Observer pe înscrieri cu un canal de notificare: atunci prefixul, tăierea și jurnalul din `ex6` se mută aproape identic.

Regula de recunoaștere: **„vreau același lucru, plus ceva, în combinații, fără să ating clasa” = Decorator. Fără contract comun, fără Decorator.**

Legătura cu ce urmează

Decorator adaugă comportament **păstrând** contractul. Cazul vecin: ai un obiect care face exact ce-ți trebuie, dar **cu altă semnătură**. Acolo intră **Adapter**, care nu adaugă nimic, doar traduce.

Harta pe online-school

Pattern	Locul	Azi?
Factory Method	`UserRepository.loadData()` :126	DA — primul, după review
Strategy	`View.play()` :38 / `playTeacher()` :232	DA — după Factory Method
Factory Method (GoF)	cele 5 `loadData()` / `save()`	mai târziu (clasă generică)
Observer	`View.registerForCourse()` :121	NU — o singură reacție
Decorator	—	NU — niciun contract comun

Document viu — crește cu fiecare lecție. Rămase: State, Adapter, Builder, Singleton.